

# Builder Journal

## 1. What I Built

I built the AI Productivity Agent, a conversational assistant that manages tasks and meetings entirely through natural language. Instead of relying on keyword matching or regex-based command parsing, I used a locally hosted LLM through Ollama, orchestrated by LangGraph, to interpret intent and reason about which tool to invoke. The stack pairs a FastAPI backend with a lightweight HTML/CSS/JavaScript frontend, LangChain for prompt formatting and model interaction, and SQLite for persistence. Core features include creating, updating, and deleting tasks, scheduling meetings, extracting action items from meeting notes, setting reminders, and answering questions about stored data, with a human-approval checkpoint before any destructive operation.

## 2. Most Difficult Technical Problem

The hardest problem was achieving reliable tool selection purely through LLM reasoning. Early on, the model would frequently choose a plausible-sounding tool that did not match the user's actual intent - for example, calling `update_task` when the user was really asking a question about a task. Because I had deliberately ruled out keyword shortcuts, I could not simply patch these cases with regex; I had to fix the reasoning itself. This was compounded by LangGraph's stateful design: a single bad tool choice early in the graph would propagate into later nodes, making failures hard to trace back to their root cause.

## 3. How I Solved It

I addressed this through iterative prompt engineering rather than code-level workarounds. I restructured the system prompt to explicitly list each tool's name, purpose, and required parameters, and added few-shot examples distinguishing similar intents, such as 'update' versus 'ask about'. I also introduced an intermediate reasoning step in the LangGraph workflow where the LLM first states its interpreted intent before selecting a tool, which I could log and inspect independently. Finally, I added schema validation on tool inputs so that malformed or incomplete arguments were rejected before execution rather than silently misexecuted.

## 4. Tool-Calling Errors Observed

During testing I observed several recurring issues: incorrect tool selection on ambiguous phrasing (e.g., 'clear my Friday' being read as deleting all tasks rather than one), missing parameters when the user omitted a due date, and occasional duplicate tool calls when the model reasoned about the same request twice within one graph execution. I handled ambiguous requests by having the agent ask a clarifying question rather than guessing, missing parameters by rejecting the call and prompting for the missing field, and duplicate executions by adding idempotency checks keyed on request identifiers within the Agent State.

## 5. Agent Behavior That Surprised Me

What surprised me most was how well the LLM handled genuinely ambiguous instructions once given adequate context. For instance, when I said 'move my 3pm to tomorrow' without specifying which meeting, the agent correctly used session memory to infer the meeting discussed earlier in the conversation, rather than failing outright. It also occasionally asked clarifying questions unprompted when confidence in intent was low, a behavior I had not explicitly programmed but that emerged from how I structured the reasoning prompt.

## 6. What Failed During Testing

Not everything worked smoothly. The agent sometimes misfired on compound requests, such as 'delete my old tasks and schedule a meeting for Monday,' executing only the first instruction. I also encountered cases where the approval workflow was bypassed because the LLM classified a bulk deletion as a single non-destructive update. Additionally, queries referencing already-deleted or non-existent meeting IDs occasionally caused unhandled database exceptions before I added existence checks, and very short or malformed inputs (e.g., a single emoji) sometimes produced confused, off-topic responses.

## 7. What I Would Redesign

In a future version, I would implement stronger planning by decomposing compound requests into an explicit multi-step plan before execution, rather than assuming one intent per turn. I would also strengthen the approval workflow to classify destructiveness at the parameter level (e.g., flagging any deletion affecting more than one record) rather than trusting a single LLM classification. Other improvements include vector-based memory for richer long-term context, structured authentication for multi-user support, and a more informative frontend that visualises pending approvals and tool execution status in real time.

## **8. What I Learned**

This project taught me that reliable agent behaviour depends far more on careful prompt design, state management, and validation than on the raw capability of the underlying model. LangGraph's explicit state model made debugging tractable by letting me inspect intermediate reasoning rather than treating the agent as a black box. I also learned that approval and error-handling logic cannot be an afterthought; they must be designed alongside tool orchestration from the outset. Overall, building this agent shifted my understanding of LLM-based systems from 'prompt and hope' to disciplined engineering with explicit safeguards.